Functional Features & Coding Solution: Development Phase Summary

Project: NeuroClass — Multimodal Attendance & Engagement Agent

Module: `5. Project Development Phase`

Phase: Implementation Summary, Codebase Directory & Technical Blueprint

---

1. Directory Structure of the Development Phase

All application source code, runtime databases, static assets, tests, and task specifications are organized within `5. Project Development Phase`:

```
5. Project Development Phase/
■■■ backend/
■   ■■■ agent/                # Autonomous Pedagogical Reasoning Agent
■   ■   ■■■ multimodal_agent.py
■   ■   ■■■ pedagogical_agent.py
■   ■■■ api/                  # FastAPI Server, WebSockets, Endpoints
■   ■   ■■■ server.py
■   ■■■ attendance/           # Sustained Presence &amp;amp; Walk-by Filtering
■   ■   ■■■ sustained_presence.py
■   ■■■ audio/                # Acoustic Feature Extraction &amp;amp; VAD
■   ■   ■■■ audio_analyzer.py
■   ■■■ core/                 # Core Vision, Scoring, Recommendations &amp;amp; Verifier
■   ■   ■■■ ai_suggestions.py
■   ■   ■■■ analytics_service.py
■   ■   ■■■ audio_analysis.py
■   ■   ■■■ camera_service.py
■   ■   ■■■ engagement_scoring.py
■   ■   ■■■ face_detection.py
■   ■   ■■■ face_recognition.py
■   ■   ■■■ face_tracker.py
■   ■   ■■■ llm_insights.py
■   ■   ■■■ pipeline_verifier.py
■   ■   ■■■ presence_attendance.py
■   ■   ■■■ recommendations_engine.py
■   ■   ■■■ teacher_profiles.py
■   ■■■ db/                   # Local Edge SQLite Database Schema &amp;amp; Manager
■   ■   ■■■ database.py
■   ■■■ fusion/               # Multimodal Temporal Synchronization
■   ■   ■■■ multimodal_fusion.py
■   ■■■ utils/                # Audio, Video, Code &amp;amp; GenAI Helpers
■   ■   ■■■ audio_utils.py
■   ■   ■■■ code_utils.py
■   ■   ■■■ genai_utils.py
■   ■   ■■■ video_generator.py
■   ■■■ vision/               # Video Ingestion, Centroid Tracker, Attention
■   ■   ■■■ attention_analyzer.py
■   ■   ■■■ face_tracker.py
■   ■   ■■■ video_stream.py
■   ■■■ config.py             # Central SystemConfig &amp;amp; Offline Rules
■   ■■■ storage.py            # Storage Management for Artifacts &amp;amp; Audio
■■■ frontend/                 # Vanilla ES6 HUD Dashboard &amp;amp; Assets
■   ■■■ css/
■   ■   ■■■ styles.css
■   ■■■ js/
■   ■   ■■■ api.js
■   ■   ■■■ app.js
■   ■■■ index.html            # Single-Page Application Real-Time HUD
■■■ student_db/               # Local Enrolled Student Face Biometrics
■■■ storage/                  # Local Exports, Generated Audio, Video Uploads
■   ■■■ attendance/
■   ■■■ audio/
■   ■■■ code/
■   ■■■ uploads/
■■■ tests/                    # 91-Test Automated System Verification Suite
■   ■■■ test_system.py
■■■ epic_specs/               # All 13 Detailed EPIC Activity Specifications
■   ■■■ EPIC_1_REQUIREMENTS.md
■   ■■■ EPIC_2_VISION_MODULE.md
■   ■■■ EPIC_2_ACTIVITY_2_2_PRESENCE_LOGIC.md
■   ■■■ EPIC_3_ACTIVITY_3_1_AUDIO_EXTRACTION.md
■   ■■■ EPIC_3_ACTIVITY_3_2_NOISE_SPEECH_ANALYSIS.md
■   ■■■ EPIC_4_ACTIVITY_4_1_MULTIMODAL_AGENT.md
```

```
■   ■■■ EPIC_4_ACTIVITY_4_2_MULTIMODAL_FUSION.md
■   ■■■ EPIC_4_ACTIVITY_4_3_ENGAGEMENT_SCORING.md
■   ■■■ EPIC_5_ACTIVITY_5_1_TEACHING_SUGGESTIONS.md
■   ■■■ EPIC_5_ACTIVITY_5_2_STUDENT_ANALYTICS.md
■   ■■■ EPIC_6_ACTIVITY_6_1_RUN_CORE_APP.md
■   ■■■ EPIC_6_ACTIVITY_6_2_STREAMLIT_DASHBOARD.md
■   ■■■ EPIC_6_ACTIVITY_6_3_VALIDATE_OUTPUTS_EDGE_CASES.md
■■■ classroom_intelligence.db # Active SQLite Edge Database
■■■ requirements.txt          # Python Package Specification
■■■ .env.example              # Environment Configuration Template
■■■ run_agent.py              # Primary Application Launcher &amp;amp; Verification CLI
■■■ streamlit_app.py          # Companion Streamlit Analytics Cockpit
■■■ validate_environment.py   # Environment &amp;amp; Dependency Verifier
```

2. Core Functional Features & Engineering Solutions

FEATURE-TO-CODE MAPPING MATRIX		
Feature Capability	Core Python Implementation Module	Architectural Highlight
1. Sustained Presence	`backend/attendance/sustained_presence.py`	Enforces 65% window ratio;
Attendance	`backend/core/presence_attendance.py`	filters walk-bys (<15s)
2. Visual Attention	`backend/vision/attention_analyzer.py`	Estimates 3D Yaw/Pitch;
& Head Pose Gaze	`backend/vision/face_tracker.py`	flags inattention > 25°
3. Privacy-Safe Audio	`backend/audio/audio_analyzer.py`	Evaluates RMS & VAD ratios;
Analysis (Zero STT)	`backend/core/audio_analysis.py`	0% speech-to-text recorded
4. Multimodal Fusion	`backend/fusion/multimodal_fusion.py`	5-second sliding window;
& Scoring Index	`backend/core/engagement_scoring.py`	Weighted 50/20/30 composite
5. Autonomous Teaching	`backend/agent/pedagogical_agent.py`	Dual-brain rule/LLM agent;
Agent Nudges	`backend/core/ai_suggestions.py`	45s anti-fatigue cooldown
6. 3-Way Hybrid	`backend/core/face_recognition.py`	File upload, live camera
Biometric Enrollment	`backend/api/server.py`	snap, and 1-click HUD link
7. Telemetry & Analytics	`backend/api/server.py`	Real-time WebSocket feed +
Cockpit	`streamlit_app.py`	Streamlit historical curves

3. Critical API Endpoints & Telemetry Gateways

Method	Endpoint Route	Description & Payload
`GET`	`/`	Serves the production single-page application (`frontend/index.html`).
`GET`	`/api/status`	Returns edge system status, offline verification flag, and active modules.
`POST`	`/api/sessions/start`	Initiates a new classroom session with course and instructor metadata.
`POST`	`/api/sessions/end`	Closes the active session and seals attendance logs.
`POST`	`/api/students/register`	Registers a student via file upload or base64 face crop.
`POST`	`/api/tracking/link-student`	**3-Way Hybrid Enrollment:** Links live track (`TRK-001`) to Student ID (`STU-101`).
`GET`	`/api/attendance/summary`	Delivers active session attendance totals, breakdown by status, and records.
`GET`	`/api/analytics/export/csv`	Exports tamper-evident official attendance CSV with timestamps.
`GET`	`/api/suggestions`	Retrieves recent pedagogical suggestions generated by the reasoning agent.
`WS`	`/ws/telemetry`	Bi-directional WebSocket streaming live HUD telemetry at 15 FPS.
`WS`	`/ws/audio`	Ingests real-time microphone PCM chunks and streams audio dynamics.

4. Execution & Verification Instructions

The complete application stack can be launched locally using the following CLI commands. All dependencies execute within the self-contained virtual environment.

Direct Execution inside `5. Project Development Phase`:

Execute the following terminal commands to launch the FastAPI server or the Streamlit analytics dashboard directly from the development phase directory:

```
# 1. Validate environment and 14 core dependencies
python validate_environment.py

# 2. Run end-to-end 5-module pipeline verification
python run_agent.py --verify-pipeline

# 3. Launch core FastAPI backend &amp;amp; Web HUD (Port 8000)
python run_agent.py

# 4. Launch companion Streamlit analytics dashboard (Port 8501)
streamlit run streamlit_app.py

# 5. Run full 91-test automated verification suite
python tests/test_system.py
```

Universal Execution from Repository Root:

```
python run_from_root.py --verify     # 5-module pipeline audit
python run_from_root.py --test       # Run 91 automated tests
python run_from_root.py --validate   # Dependency verification
python run_from_root.py --agent      # Launch core FastAPI server
python run_from_root.py --streamlit  # Launch Streamlit dashboard
```